

GROUP LAB ACTIVITY - RTL WISH 2026 – TI

May 26, 2026

8 -BIT SEQUENTIAL SHIFT AND ADD MULTIPLIER

GOAL:

You need to design an **8-bit sequential shift and add multiplier** as a group of 5 students. Multiplier should consist of **5 different modules** as described so that each student builds one module.

Final module should compute 16-bit product over multiple cycles using a shift-and-add datapath plus a control FSM.

How to test?

A testbench file("tb_seq_multiplier.v") is provided to you. Run this testbench along with your Verilog files.

This testbench should be used to test your sequential multiplier. It applies a few test cases and checks the results automatically, printing PASS or FAIL messages. You can add more test cases by calling the run task with different values of a and b. You can also view the waveforms in GTKWave by opening the generated multiplier_tb_gtk.vcd file.

```
..\student> iverilog -o multiplier_tb.vvp *  
..\student> vvp multiplier_tb.vvp  
..\student> gtkwave multiplier_tb_gtk.vcd
```

Module Description

Student	Module	Input	Output	Job
1	mult_regs	clk, rst, load, add_en, sum, shift, a_in, b_in	A, M, Q	Store and manipulate registers
2	bit_counter	clk, rst, clear, count_en	count, last	Count 0→7, signal when done
3	add_decision	A, M, Q_lsb	sum, do_add	Compute A+M, flag for add
4	controller	clk, rst, start, last, do_add	load, add_en, shift, count_en, clear, done	Orchestrate FSM, drive control signals
5	seq_multiplier	clk, rst, start, a, b	product, done	Wire modules, present interface

Detailed Description

Student 1: Datapath Registers (mult_regs)

Module Name: mult_regs

Purpose: Stores and manipulates the three key registers: M (multiplicand), Q (multiplier), and A (accumulator).

Port Specification

Port	Width	Description
Inputs		
`clk`	1	Clock signal. All changes happen on rising edge
`rst`	1	Active-HIGH reset When rst=1, clear all registers(A,M,Q). When rst=0, normal operation
`load`	1	Load enable When load=1, latch the input values a_in and b_in into M and Q, and clear A
`sum[8:0]`	9	Stores current value of $A + \{1'b0, M\}$
`add_en`	1	Add enable When add_en=1, loads sum[8:0] into A
`shift`	1	Shift enable When shift=1, right-shift the 17-bit combined register {A,Q} by 1 bit
`a_in[7:0]`	8	Input multiplicand (loaded into M when load=1)
`b_in[7:0]`	8	Input multiplier (loaded into Q when load=1)
Outputs		
`A[8:0]`	9	Accumulator register (9 bits to hold carry from addition). Current partial sum
`M[7:0]`	8	Multiplicand register (stays constant during multiply)
`Q[7:0]`	8	Multiplier/Product register (shifts right, gradually becomes product)

Key Points

Reset has highest priority. Then load, add, shift in order. Only ONE operation per cycle.

Think about this:

- Why is A 9 bits, not 8? Walk through 255×255 .
- Why is the 17-bit shift over {A,Q} cheaper than two separate shifts?
- What if add_en and shift are asserted in the same cycle?
- Do you reset M and Q, or only A? Defend the choice.

Student 2: Bit Counter (bit_counter)

Module Name: bit_counter

Purpose: Counts from 0 to 7 over 8 cycles. Signals the FSM when we've completed 8 shifts (multiplication done).

Port Specifications

Port	Width	Description
Inputs		
`clk`	1-bit	Clock signal. Count updates on rising edge.
`rst`	1-bit	Active-HIGH reset: When rst=1, count $\leftarrow$ 0.
`clear`	1-bit	Clear enable (synchronous): On rising edge of clk, if clear=1, set count $\leftarrow$ 0.
`count_en`	1-bit	Count enable: On rising edge of clk, if count_en=1, increment count by 1.
Outputs		
`count[3:0]`	4-bit	Current counter value (0 to 7). Used by FSM to track cycles.
`last`	1-bit	Combinational flag: Asserts (=1) when count==7. Tells FSM: "This is the last shift cycle."

Key Points

Reset has highest priority. Then clear, count_en in order. Only ONE operation per cycle.

Think about this:

- Should 'last' be registered or combinational? How does it align with the FSM?
- Up-count vs down-count to zero — which is cheaper, and why?
- If clear and count_en are both high, define priority in words first.

Student 3: Conditional Add Logic (add_decision)

Module Name: add_decision

Purpose: Purely **combinational logic** that computes the sum $A+M$ and outputs a flag indicating whether to add (based on $Q[0]$).

Port Specifications

Port	Width	Description
Inputs		
<code>`A[8:0]`</code>	9-bit	Current accumulator value from mult_regs.
<code>`M[7:0]`</code>	8-bit	Multiplicand from mult_regs.
<code>`Q_lsb`</code>	1-bit	Least significant bit of Q ($Q[0]$). Tells us: "Should we add?"
Outputs		
<code>`sum[8:0]`</code>	9-bit	Result of $A + \{1'b0, M\}$. Always computed , regardless of <code>Q_lsb</code> .
<code>`do_add`</code>	1-bit	Flag output. Equal to <code>Q_lsb</code> . Tells the FSM: "Use this sum if <code>do_add=1</code> ."

Key Points

This is pure combinational logic. No registers, no reset.

Think about this:

- Why compute sum unconditionally and gate the write instead of gating the adder input?
- Why is this a separate module from Student 1? Defend on testability grounds.

Student 4: Control FSM (controller)

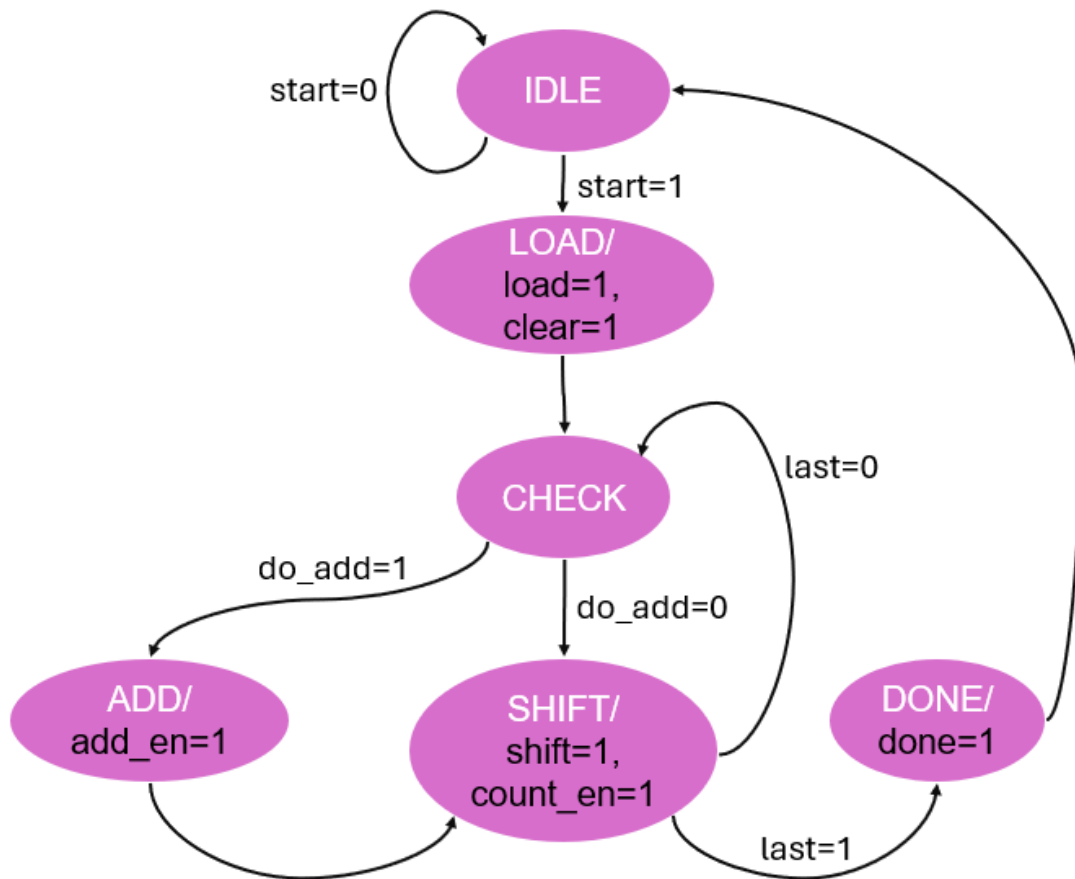
Module Name: controller

Purpose: State machine that orchestrates all operations: decides when to LOAD, when to ADD, when to SHIFT, and when to signal DONE.

Port Specifications

Port	Width	Description
Inputs		
<code>`clk`</code>	1-bit	Clock. State updates on rising edge.
<code>`rst`</code>	1-bit	Active-HIGH asynchronous reset: Return to IDLE state.
<code>`start`</code>	1-bit	Pulse input (high for 1 cycle): Testbench asserts this to begin a new multiply. FSM transitions IDLE→LOAD.
<code>`last`</code>	1-bit	Input from bit_counter: High when counter==7. Tells FSM: "8 shifts done, exit on next clock."
<code>`do_add`</code>	1-bit	Input from add_decision: Tells FSM: "Should we add?" (1=add, 0=skip).
Outputs		
<code>`load`</code>	1-bit	To mult_regs: Asserts when FSM is in LOAD state. Tells datapath: "Load new inputs."

<code>`add_en`</code>	1-bit	To mult_regs: Asserts when FSM is in ADD state. Tells datapath: "Add M to A."
<code>`shift`</code>	1-bit	To mult_regs: Asserts when FSM is in SHIFT state. Tells datapath: "Shift {A, Q}."
<code>`count_en`</code>	1-bit	To bit_counter: Asserts when FSM is in SHIFT state. Tells counter: "Increment."
<code>`clear`</code>	1-bit	To bit_counter: Asserts when FSM is in LOAD state. Tells counter: "Reset to 0."
<code>`done`</code>	1-bit	To testbench: Asserts when FSM is in DONE state. Tells testbench: "Result is ready."



State Diagram

Key Points

Timing: FSM transitions on rising edge; outputs are available combinationaly (same cycle)

Think about this:

- *Two-always vs three-always FSM coding — pick and justify.*
- *Off-by-one: does the FSM exit ON cycle 7 or AFTER? Prove correctness.*
- *One-hot vs binary — when does it start to matter?*

Student 5: Top-Level Multiplier (seq_multiplier)

Module Name: seq_multiplier

Purpose: Instantiate and wire together all 4 modules (mult_regs, bit_counter, add_decision, controller) and the testbench.

Port Specifications

Port	Width	Description
Inputs		
`clk`	1-bit	System clock. All modules synchronize to this.
`rst`	1-bit	Active-HIGH reset: Resets all sub-modules.
`start`	1-bit	Pulse from testbench (1 cycle high): Initiates a new multiply.
`a[7:0]`	8-bit	Multiplicand (first operand). Valid when start is asserted.
`b[7:0]`	8-bit	Multiplier (second operand). Valid when start is asserted.
Outputs		
`product[15:0]`	16-bit	The multiplication result ($a \times b$). Valid when done=1.
`done`	1-bit	Completion flag: Asserts (=1) when product is ready. Testbench waits for this.

Key Points

Timing: FSM transitions on rising edge; outputs are available combinationaly (same cycle)

Think about this:

- *How long is 'product' valid after done? Document the choice.*